

MIT Open Access Articles

Better Defunctionalization through Lambda Set Specialization

The MIT Faculty has made this article openly available. **Please share** how this access benefits you. Your story matters.

Citation: Brandon, William, Driscoll, Benjamin, Dai, Frank, Berkow, Wilson and Milano, Mae. 2023. "Better Defunctionalization through Lambda Set Specialization." Proceedings of the ACM on Programming Languages, 7 (PLDI).

As Published: <https://doi.org/10.1145/3591260>

Publisher: ACM

Persistent URL: <https://hdl.handle.net/1721.1/150980>

Version: Final published version: final published article, as it appeared in a journal, conference proceedings, or other formally published context

Terms of use: Creative Commons Attribution





Better Defunctionalization through Lambda Set Specialization

WILLIAM BRANDON*, Massachusetts Institute of Technology, USA

BENJAMIN DRISCOLL*, Stanford University, USA

FRANK DAI, WILSON BERKOW, and MAE MILANO, University of California, Berkeley, USA

Higher-order functions pose a challenge for both static program analyses and optimizing compilers. To simplify the analysis and compilation of languages with higher-order functions, a rich body of prior work has proposed a variety of *defunctionalization* techniques, which can eliminate higher-order functions from a program by transforming the program to a semantically-equivalent first-order representation. Several modern languages take this a step further, *specializing* higher-order functions with respect to the functions on which they operate, and in turn allowing compilers to generate more efficient code. However, existing specializing defunctionalization techniques restrict how function values may be used, forcing implementations to fall back on costly dynamic alternatives. We propose *lambda set specialization* (LSS), the first specializing defunctionalization technique which imposes no restrictions on how function values may be used. We formulate LSS in terms of a polymorphic type system which tracks the flow of function values through the program, and use this type system to recast specialization of higher-order functions with respect to their arguments as a form of type monomorphization. We show that our type system admits a simple and tractable type inference algorithm, and give a formalization and fully-mechanized proof in the Isabelle/HOL proof assistant showing soundness and completeness of the type inference algorithm with respect to the type system. To show the benefits of LSS, we evaluate its impact on the run time performance of code generated by the MLton compiler for Standard ML, the OCaml compiler, and the new Morphic functional programming language. We find that pre-processing with LSS achieves run time speedups of up to 6.85× under MLton, 3.45× for OCaml, and 78.93× for Morphic.

CCS Concepts: • **Software and its engineering** → *Polymorphism*; **Procedures, functions and subroutines**.

Additional Key Words and Phrases: monomorphization, defunctionalization, type systems

ACM Reference Format:

William Brandon, Benjamin Driscoll, Frank Dai, Wilson Berkow, and Mae Milano. 2023. Better Defunctionalization through Lambda Set Specialization. *Proc. ACM Program. Lang.* 7, PLDI, Article 146 (June 2023), 24 pages. <https://doi.org/10.1145/3591260>

1 INTRODUCTION

Higher-order functions are ubiquitous and desirable, but challenging to implement efficiently. Dynamic function values inhibit inlining and pose an obstacle to interprocedural analysis. Moreover, their implementations frequently rely on heap-allocated storage and indirect jumps, imposing runtime overhead compared to calling a known function symbol.

*Both authors contributed equally to this research.

Authors' addresses: William Brandon, wbrandon@csail.mit.edu, Massachusetts Institute of Technology, USA; Benjamin Driscoll, bdrisc@stanford.edu, Stanford University, USA; Frank Dai, fydai@berkeley.edu; Wilson Berkow, wberkow@berkeley.edu; Mae Milano, mpmilano@berkeley.edu, University of California, Berkeley, USA.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2023 Copyright held by the owner/author(s).

2475-1421/2023/6-ART146

<https://doi.org/10.1145/3591260>

These challenges have motivated the need for *defunctionalization* transformations: algorithms which transform a higher-order program into an equivalent first-order representation. Dating back to the first defunctionalization technique proposed by Reynolds [1972], defunctionalization transformations have traditionally been *monovariant*, meaning that each higher-order function is transformed to *at most one* first-order function. Unfortunately, monovariant defunctionalization can produce output which is difficult to analyze and difficult to compile to efficient code, because each higher-order function must be transformed to a first-order function embodying *all* its possible behaviors across all call sites in the program.

In contrast to traditional monovariant techniques, some more recent approaches to defunctionalization employ *specialization*, allowing a single higher-order function to give rise to multiple first-order functions specialized to different call sites. Specializing defunctionalization promises a dramatic increase in the effectiveness of downstream optimizations by reducing or eliminating jump tables. Furthermore, unlike monovariant approaches, specializing defunctionalization need not lose precision when repeated expressions are factored out into reusable functions.

Restricted versions of specializing defunctionalization, such as C++ and Rust’s aggressive monomorphizing strategy and Julia JIT-enhanced approach, are already in widespread use [Bezan-son et al. 2017; Hovgaard et al. 2018; Klabnik and Nichols 2019, Ch. 10; Paszke et al. 2021; Stroustrup 2013, Ch. 26]. However, all existing specializing techniques suffer from limitations of *expressiveness*—either not supporting truly first-class functions (as in [Hovgaard et al. 2018; Paszke et al. 2021]), or forcing the programmer to fall back to traditional, slow virtually-dispatched closures in the general case (as in C++ and Rust [Klabnik and Nichols 2019, Ch. 13; Stroustrup 2013, Ch. 11]).

In this work we introduce *lambda set specialization* (LSS), the first *specializing* defunctionalization capable of eliminating *all* higher-order control flow in a realistic functional source language. Unlike existing industry-standard specializing techniques, LSS has no expressiveness limitations, and never falls back to traditional dynamic function representations. In our evaluation, we find that the ability of LSS to specialize higher-order functions to their call sites produces significantly more efficient code than the state-of-the-art monovariant defunctionalization of MLton [Cejtin et al. 2000], yielding a speedup of up to 6.85× over MLton’s built-in defunctionalization.

We formulate LSS in terms of a polymorphic type system which we use to track the flow of function values through the program. In this type system, each function type is decorated with a *lambda set annotation*, which symbolically tracks the set of possible lambdas it might contain at runtime. Using this type information, we are able to perform specialization by recasting specialization of higher-order functions as a form of *type monomorphization*. To apply LSS to arbitrary source programs without lambda set annotations, we have developed a type inference algorithm capable of converting any well-typed program in our source language to a well-typed program in our extended type system. Using Isabelle/HOL, we give a fully mechanized proof of the soundness and completeness of this inference algorithm with respect to our type system.

The main contributions of this paper are as follows:

- (1) We introduce *lambda set specialization*, a new defunctionalization technique which achieves the performance benefits of specializing defunctionalization while maintaining the expressiveness of prior non-specializing approaches.
- (2) We formulate LSS in terms of type inference and type monomorphization within a novel polymorphic type system.
- (3) We formally prove in Isabelle/HOL that our type inference is sound and complete with respect to our type system.
- (4) We conduct a comprehensive evaluation of the performance impact of LSS, showing speedups of up to 6.85× for code compiled under MLton, 3.45× for OCaml, and 78.93× for Morp hic.

2 OVERVIEW

To motivate our transformation, consider the following pseudocode:

```
def twice(f : int → int, x : int) → int = f(f(x))
let y = twice((λx. x + 2), 5)
let maybeplus2 = if b then λx. x else λx. x + 2
let z = twice(maybeplus2, 5)
```

Suppose we would like to transform this program to a first-order representation—that is, to defunctionalize it. In particular, we would like to transform the higher-order function *twice* into first-order code whose type annotations and control flow are as simple as possible. Using a *specializing* strategy to achieve this goal, we might hope to arrive at something like the following:

```
def plus2(x : int) → int = x + 2
def id(x : int) → int = x
def twice_plus2(x : int) → int = plus2(plus2(x))
def twice_maybeplus2(tag : ('id | 'plus2), x : int) → int =
  match tag { 'id → id(id(x)), 'plus2 → plus2(plus2(x)) }
let y = twice_plus2(5)
let maybeplus2 = if b then 'id else 'plus2
let z = twice_maybeplus2(maybeplus2, 5)
```

In this transformation, we have generated a new top-level function for each lambda in the program, and two separate top-level functions for *twice*: *twice_plus2* and *twice_maybeplus2*. Each of these two new *twice* functions has been specialized for a specific syntactic call site of *twice*, with the function's body modified to directly call the possible lambdas that might be passed at that call site.

Thanks to specialization, each transformed version of *twice* precisely characterizes the control flow possible at its particular call site. Importantly, *twice_plus2* is able to dispatch statically to a known jump target. By contrast, the jump target of *twice_maybeplus2* is runtime dependent, with its dynamic lambda argument represented by a *sum type* whose possible variants are 'id and 'plus2. The idea of representing sets of possible lambdas by sum types is standard in the defunctionalization literature [Cejtin et al. 2000; Reynolds 1972]. However, such *lambda-set-based* defunctionalization has traditionally been monovariant, rather than specializing. A compiler employing monovariant defunctionalization, such as MLton, would merge the functions *twice_plus2* and *twice_maybeplus2* into a single function. We seek a *specializing* technique which avoids conflating the behavior of higher-order functions at independent call sites.

Interestingly, a form of specializing defunctionalization is already in widespread use in Rust and C++, which both support the use of templating mechanisms to instantiate higher-order functions independently for different individual lambdas. However, existing examples of such templating-based approaches lack support for specializing with respect to lambda sets, such as ('id | 'plus2) above, and would instead need to fall back to an opaque virtually-dispatched implementation of lambdas to represent a variable like *maybeplus2*.

With our technique, we aim to capture the best of both approaches: defunctionalizing *without restrictions* on how lambdas may be used, while still *specializing* for efficiency. To do this, we must perform a precise, decidable, call-site specific flow analysis in the presence of arbitrary higher-order functions, recursive functions, and recursive datatypes.

Our key idea is to formulate our analysis in terms of a *polymorphic type system* which tracks the flow of function values through the program. In this type system, we decorate every function type with a *lambda set annotation*. A lambda set annotation may consist of a concrete set of possible lambdas, such as $\{\lambda x.x, \lambda x.x + 2\}$, but is also allowed to take the form of a symbolic *lambda set variable*. Using lambda set variables, it is possible for higher-order functions to be *polymorphic* over lambda sets, which in turn allows lambda set annotations to vary between call sites. We return to our example, now decorated with lambda set annotations:

```

def  $\text{twice}\langle\alpha\rangle(f : \mathbf{int} \xrightarrow{\alpha} \mathbf{int}, x : \mathbf{int}) \rightarrow \mathbf{int} = f(f(x))$ 
let  $y : \mathbf{int} = \text{twice}\langle\{\lambda x. x + 2\}\rangle(\lambda x. x + 2, 5)$ 

let  $\text{maybeplus2} : \mathbf{int} \xrightarrow{\{\lambda x. x, \lambda x. x+2\}} \mathbf{int} = \text{if } b \text{ then } \lambda x. x \text{ else } \lambda x. x + 2$ 
let  $z = \text{twice}\langle\{\lambda x. x, \lambda x. x + 2\}\rangle(\text{maybeplus2}, 5)$ 

```

Note that the type of *twice* is now polymorphic over a lambda set parameter α , which it uses to refine the type $\mathbf{int} \rightarrow \mathbf{int}$ to $\mathbf{int} \xrightarrow{\alpha} \mathbf{int}$. In different calls to *twice*, the parameter α can take on different concrete lambda set values, capturing the information required to specialize *twice*. In the definition of *y*, we see that the lambda set $\{\lambda x. x + 2\}$ constrains *twice* to take as its first argument exactly the lambda $\lambda x. x + 2$. In the definition of *maybeplus2*, we see that the lambda set annotation on its type indicates that it must be either $\lambda x. x$ or $\lambda x. x + 2$. This matches the lambda set argument to the application of *twice* in the definition of *z*, ensuring that *twice* is specialized to exactly the lambdas that may reside in *maybeplus2*.

Using such an extended type system, our *lambda set specialization* technique proceeds in three passes: a *type inference* pass, a *specialization* pass, and a *lowering* pass. Given a well-typed source program without lambda set annotations, our type inference elaborates it to a well-typed program in our extended type system. We take care to infer signatures which are polymorphic over lambda sets, to facilitate specialization. During the specialization pass, we *monomorphize* definitions with respect to our inferred types, instantiating specialized copies of polymorphic definitions as necessary to ensure that all lambda set annotations become concrete. In the lowering pass, we finally use the program's lambda set annotations to translate it to a first-order representation, converting function types to sum types, and function applications to pattern matches on those sum types.

3 SETTING: THE L^{src} LANGUAGE

We present our technique, LSS, in the setting of a small higher-order functional language L^{src} , an extension of the simply-typed lambda calculus. In addition to function types, we equip L^{src} with n -ary product and sum types, as well as isorecursive μ types [Pierce 2002]. The **inj** and **match** constructs introduce and eliminate elements of sum types, while **fold** and **unfold** introduce and eliminate elements of μ types. The full syntax of L^{src} is given in Figure 1. Our examples use a sugared version of L^{src} which includes integer types, tuple destructuring, and implicit lambda captures, and leave some type annotations implicit where they are unambiguous.

To guide specialization, we include in L^{src} a notion of *top-level definitions* distinguished from the local bindings introduced by **let**, **match**, and lambda abstraction. Top-level definitions are introduced with **def** and are intended to act as the *unit of specialization*. Consider the L^{src} program

```

def  $\text{twice} = \lambda(f : \mathbf{int} \rightarrow \mathbf{int}, x : \mathbf{int}). f(f(x))$  in
  ( $\text{twice}(\lambda(x : \mathbf{int}). x + 1, 5), \text{twice}(\lambda(x : \mathbf{int}). x - 1, 5)$ )

```

Because **def** bindings act as the unit of specialization, this program would defunctionalize to a first-order program where the **def**-bound function *twice* is specialized into two distinct functions,

$x, y \in \text{Var} \quad d \in \text{Def-Name} \quad a \in \text{Mu-Var} \quad i, n \in \mathbb{N}$
 $t \in \text{Type} ::= t_1 \rightarrow t_2 \mid t_1 \times \cdots \times t_n \mid t_1 + \cdots + t_n \mid \mu a. t \mid a$
 $e \in \text{Expr} ::= (e_1, \dots, e_n) \mid \mathbf{proj}_i(e) \mid \mathbf{inj}_{t_1 + \dots + t_n}^i(e) \mid \mathbf{fold}_{\mu a. t}(e) \mid \mathbf{unfold}(e) \mid \mathbf{let} \, x = e_1 \, \mathbf{in} \, e_2$
 $\quad \mid \mathbf{match} \, e \{ \mathbf{inj}^1(x_1) \rightarrow e_1, \dots, \mathbf{inj}^n(x_n) \rightarrow e_n \} \mid \lambda[x : t_1 = e_1](y : t_2). (e_2 : t_3) \mid e_1(e_2) \mid x \mid d$
 $p \in \text{Program} ::= d \mid \mathbf{def} \, d : t = e \, \mathbf{in} \, p$

Fig. 1. The L^{src} Language

one for each lambda with which it was invoked. By contrast, if *twice* had been **let**-bound, as in:

$$\begin{aligned}
 \mathbf{let} \, twice &= \lambda(f : \mathbf{int} \rightarrow \mathbf{int}, x : \mathbf{int}). f(f(x)) \, \mathbf{in} \\
 &(\mathbf{twice}(\lambda(x : \mathbf{int}). x + 1, 5), \mathbf{twice}(\lambda(x : \mathbf{int}). x - 1, 5))
 \end{aligned}$$

then *twice* would not be specialized, and would instead transform after defunctionalization to a single function capable of accommodating both lambdas.

Since specialization may duplicate a **def** binding, or even delete it if it is unused, the semantics of **def** bindings with side effects must be given special consideration. To sidestep this issue, L^{src} does not evaluate **def** bindings where they are defined, but rather reevaluates them each time they are used. In a real-world setting, one might alternatively choose to enforce a constraint on **def** bindings similar to ML's value restriction [Wright 1995] to ensure that they can be safely duplicated.

We denote the big-step operational semantics of L^{src} with a judgment of the form $\Delta ; \Gamma \vdash e \Downarrow v$. Here, v is a metavariable ranging over runtime *values* drawn from the following set:

$$v \in \text{Value} ::= \lambda[x = v]y.e \mid (v_1, \dots, v_n) \mid \mathbf{inj}^i(v)$$

The full operational semantics of L^{src} can be found in Appendix A. We use Δ to refer to the context of **def** bindings ($d := e$) and Γ to refer to the context of local bindings ($x := e$). A **def** binding may refer to earlier **def** bindings and to itself (allowing recursive L^{src} programs), but not to later **def** bindings. An L^{src} program must always end with the name of a single **def** to use as the program's *entry point*. Evaluating the program then consists of evaluating the entry point. In examples, as shorthand, we often end programs with a full expression e instead.

To simplify our formal treatment, we allow lambda abstractions in L^{src} to refer to any **def**-bound variables in scope, but *not* to any local variables. Instead, for a lambda abstraction to capture values from its environment it must specify an explicit *capture expression*, written with the notation $[x : \dots = \dots]$. In the following example, the L^{src} **def** *make-mult* returns a lambda capturing n :

$$\mathbf{def} \, \mathbf{make-mult} : \mathbf{int} \rightarrow (\mathbf{int} \rightarrow \mathbf{int}) = \lambda(n : \mathbf{int}). (\lambda[c : \mathbf{int} = n](x : \mathbf{int}). x * c)$$

For a lambda abstraction $\lambda[x : t_1 = e_1](y : t_2).e_2$, the expression e_1 is evaluated at the point where the lambda is constructed and stored in the captured variable x , and the only local variables which may appear free in e_2 are the capture x and the argument y . Without loss of generality, lambdas with no captures can be modeled as capturing **unit**, and lambdas with multiple captures can be modeled as capturing a tuple. Similarly, lambdas with multiple arguments can be modeled as lambdas taking a single tuple argument. A language with implicit lexical captures can be converted to this form by a straightforward transformation. The full typing rules for L^{src} are given in Appendix B.

4 LAMBDA SET ANALYSIS AS POLYMORPHIC TYPE INFERENCE

At the core of our LSS technique is an intermediate language L^{annot} . It features a type system capable of modeling all the lambda set information necessary to perform specializing defunctionalization.

$$\begin{aligned}
\alpha, \beta, \gamma &\in \text{Lam-Set-Var} & \ell \in \text{Lambda} &::= \lambda[x : \tau_1](y : \tau_2).(\epsilon : \tau_3) \\
\sigma \in \text{Lambda-Set} &::= \{\ell_1, \dots, \ell_n\} \mid \alpha \mid \mu a. \sigma \mid a \\
\tau \in \text{Annot-Type} &::= \tau_1 \xrightarrow{\sigma} \tau_2 \mid \tau_1 \times \dots \times \tau_n \mid \tau_1 + \dots + \tau_n \mid \mu a. \tau \mid a \\
\epsilon \in \text{Annot-Expr} &::= (\epsilon_1, \dots, \epsilon_n) \mid \mathbf{proj}_i(\epsilon) \mid \mathbf{inj}_{\tau_1 + \dots + \tau_n}^i(\epsilon) \mid \mathbf{fold}_{\mu a. \tau}(\epsilon) \mid \mathbf{unfold}(\epsilon) \\
&\mid \mathbf{let} x = \epsilon_1 \mathbf{in} \epsilon_2 \mid \mathbf{match} \epsilon \{ \mathbf{inj}^1(x_1) \rightarrow \epsilon_1, \dots, \mathbf{inj}^n(x_n) \rightarrow \epsilon_n \} \\
&\mid (\lambda[x : \tau_1 = \epsilon_1](y : \tau_2).(\epsilon_2 : \tau_3)) \mathbf{as} \sigma \mid (\epsilon_1 \mathbf{as} \sigma)(\epsilon_2) \mid x \mid d\langle \sigma_1, \dots, \sigma_n \rangle \\
C \in \text{Constraint} &::= \ell \in \sigma & Q \in \text{Constraints} &::= \{C_1, \dots, C_n\} \\
\pi \in \text{Annot-Program} &::= d\langle \rangle \mid \mathbf{def} d\langle \alpha_1, \dots, \alpha_n \rangle : (Q \Rightarrow \tau) = \epsilon \mathbf{in} \pi
\end{aligned}$$
Fig. 2. The L^{annot} Language

The L^{annot} language is an extension of L^{src} which places *lambda set annotations* on certain types and expressions. The full syntax of L^{annot} is presented in Figure 2. Note that annotated types and expressions in L^{annot} are referred using Greek variables (τ, ϵ), to distinguish them from their unannotated equivalents (t, e) in L^{src} . The languages L^{src} and L^{annot} are related by an *erasure operator* $[\cdot]$, which takes any L^{annot} term and strips it of its lambda set annotations, yielding an L^{src} term. The full definition of $[\cdot]$ is given in Appendix E. We define the operational semantics of an L^{annot} term to be the operational semantics of its erasure.

We introduce lambda set annotations in three key places, each motivated by a different requirement of the defunctionalization transformation:

- Function types in L^{annot} take the form $\tau_1 \xrightarrow{\sigma} \tau_2$, where σ is a lambda set representing an upper bound on the possible lambdas which might inhabit the function type at runtime. This tracks the information needed to transform function types into sum types.
- Function applications in L^{annot} take the form $(\epsilon_1 \mathbf{as} \sigma)(\epsilon_2)$, where σ is the lambda set of the function ϵ_1 . This tracks the information needed to transform function applications into pattern matches.
- Lambda abstractions in L^{annot} take the form $(\lambda \dots) \mathbf{as} \sigma$, where σ is a “target” lambda set large enough to accommodate both the lambda being constructed *and* any other lambdas which might be present at locations to which the result of the current expression might flow. This tracks the information needed to transform abstractions to injections into sum types.

As described so far, these lambda set annotations are conceptually similar to the whole-program data flow annotations collected as part of existing non-specializing defunctionalization techniques, like that of MLton [Cejtin et al. 2000]. However, in order to support specialization, we break from prior work by introducing a mechanism for *lambda set polymorphism* into the L^{annot} type system. In the next section we discuss the design of this polymorphism mechanism in more detail.

4.1 Lambda Set Polymorphism and Inclusion Constraints

Recall that top-level **def** bindings in L^{src} are intended to act as the *unit of specialization* during defunctionalization, each potentially giving rise to multiple **def** bindings in the defunctionalized program. Therefore, we introduce *lambda set polymorphism* at the level of **def** bindings. A **def** binding in L^{annot} may be parameterized by zero or more *lambda set variables*, which may appear in the binding’s type and body in place of concrete lambda set annotations.

We write the lambda set parameters for a definition in angle brackets $\langle \cdot \cdot \cdot \rangle$ after its name, both when defining a **def** binding and when calling it. For instance, we could lift the L^{src} definition from

Expression Typing:

$$\boxed{\Delta; Q; \Gamma \vdash \epsilon : \tau}$$

$$\frac{\ell = \lambda[x : \tau_1](y : \tau_2).(\epsilon_2 : \tau_3) \quad Q \vdash \ell \in \sigma \quad \Delta; Q; \Gamma \vdash \epsilon_1 : \tau_1 \quad \Delta; Q; \{x : \tau_1, y : \tau_2\} \vdash \epsilon_2 : \tau_3}{\Delta; Q; \Gamma \vdash \ell[\epsilon_1] \text{ as } \sigma : \tau_2 \xrightarrow{\sigma} \tau_3} [\text{AT-Abs}]$$

$$\frac{\Delta; Q; \Gamma \vdash \epsilon_1 : \tau_1 \xrightarrow{\sigma} \tau_2 \quad \Delta; Q; \Gamma \vdash \epsilon_2 : \tau_1}{\Delta; Q; \Gamma \vdash (\epsilon_1 \text{ as } \sigma)(\epsilon_2) : \tau_2} [\text{AT-APP}] \quad \frac{(d : \forall \bar{\alpha}. Q_2 \Rightarrow \tau) \in \Delta \quad Q_1 \vdash Q_2[\bar{\alpha} \mapsto \sigma]}{\Delta; Q_1; \Gamma \vdash d(\bar{\sigma}) : \tau[\bar{\alpha} \mapsto \sigma]} [\text{AT-DEF-REF}]$$

Program Typing:

$$\boxed{\Delta \vdash \pi : \tau}$$

free lambda set variables of $Q, \tau_1, \epsilon \subseteq \{\bar{\alpha}\}$

$$\frac{\Delta \cup \{d : \forall \bar{\alpha}. Q \Rightarrow \tau_1\}; Q; \emptyset \vdash \epsilon : \tau_1 \quad \Delta \cup \{d : \forall \bar{\alpha}. Q \Rightarrow \tau_1\} \vdash \pi : \tau_2 \quad d \notin \Delta}{\Delta \vdash \text{def } d(\bar{\alpha}) : (Q \Rightarrow \tau_1) = \epsilon \text{ in } \pi : \tau_2} [\text{AT-DEF}] \quad \frac{(d : \emptyset \Rightarrow \tau) \in \Delta \quad \tau \text{ has no function types}}{\Delta \vdash d\langle \rangle : \tau} [\text{AT-ENTRY}]$$

Constraint Entailment:

$$\frac{(\ell \in \alpha) \in Q}{Q \vdash \ell \in \alpha} \quad \frac{\boxed{Q \vdash C}}{Q \vdash \ell \in \{\ell_1, \dots, \ell_n\}} \quad \frac{Q \vdash \ell \in \sigma[a \mapsto \mu a. \sigma]}{Q \vdash \ell \in \mu a. \sigma} \quad \frac{\boxed{Q_1 \vdash Q_2}}{Q \vdash \{C_1, \dots, C_n\}}$$

Fig. 3. Selected Typing Rules for L^{annot}

the *twice* example in Section 3 to the polymorphic L^{annot} definition¹

def *twice* $\langle \alpha \rangle : ((\text{int} \xrightarrow{\alpha} \text{int}), \text{int}) \rightarrow \text{int} = \lambda(f : \text{int} \xrightarrow{\alpha} \text{int}, x : \text{int}). (f \text{ as } \alpha)((f \text{ as } \alpha)(x))$

and instantiate it at its first call site by providing the concrete lambda set $\{\lambda(x : \text{int}). x + 1\}$ as α :

twice $\langle \{\lambda(x : \text{int}). x + 1\} \rangle((\lambda(x : \text{int}). x + 1) \text{ as } \{\lambda(x : \text{int}). x + 1\}, 5)$

A difficulty arises when considering the interaction between lambda set polymorphism and the lambda abstraction construct. As a central design objective, we would like L^{annot} to satisfy the property that whenever an L^{annot} program is well-typed, its lambda set annotations can be used to successfully specialize and defunctionalize it to a semantically equivalent first-order program. To this end, it is critical that whenever a lambda set σ appears as the target of a lambda abstraction, the set σ contains at least the lambda currently being constructed. However, this implies it is not safe in general to instantiate a polymorphic L^{annot} definition at arbitrary concrete lambda sets.

For example, consider the following attempt at lifting the example definition *make-mult* from Section 3 into the L^{annot} type system:

def *make-mult* $\langle \alpha \rangle : \text{int} \rightarrow (\text{int} \xrightarrow{\alpha} \text{int}) = \lambda(n : \text{int}). ((\lambda[c : \text{int} = n](x : \text{int}). x * c) \text{ as } \alpha)$

It is not safe to instantiate this definition with arbitrary concrete lambda sets in place of α . For example, the substitution $\alpha \mapsto \{\lambda(x : \text{int}). x + 1\}$ would produce the specialized definition:

¹Here we omit the lambda set annotation on the lambda abstraction for the *twice* function itself.

$$\mathbf{def\ make-mult}_{\text{spec}} \langle \rangle : \mathbf{int} \rightarrow (\mathbf{int} \xrightarrow{\{\lambda(x:\mathbf{int}). x+1\}} \mathbf{int}) =$$

$$\lambda(n : \mathbf{int}). ((\lambda[c : \mathbf{int} = n](x : \mathbf{int}). x * c) \mathbf{as} \{\lambda(x : \mathbf{int}). x + 1\})$$

which is impossible to defunctionalize. The sum type generated to represent the function type

$$\mathbf{int} \xrightarrow{\{\lambda(x:\mathbf{int}). x+1\}} \mathbf{int}$$

would not contain a variant suitable for representing $(\lambda[c : \mathbf{int} = n](x : \mathbf{int}). x * c)$.

To enforce that situations like this do not occur in well-typed L^{annot} programs, we introduce a notion of *lambda inclusion constraints* into the L^{annot} type system, which allow polymorphic L^{annot} definitions to impose constraints on the lambda sets with which they may be instantiated. We write $\ell \in \sigma$ to denote the constraint that a lambda set σ must include a lambda ℓ , and use the notation “ $Q \Rightarrow \dots$ ” to denote the *constraint set* Q of a polymorphic definition. The problematic definition above could be typed correctly using an inclusion constraint as:

$$\mathbf{def\ make-mult} \langle \alpha \rangle : (\{(\lambda[c : \mathbf{int}](x : \mathbf{int}). x * c) \in \alpha\} \Rightarrow \mathbf{int} \rightarrow (\mathbf{int} \xrightarrow{\alpha} \mathbf{int})) =$$

$$\lambda(n : \mathbf{int}). ((\lambda[c : \mathbf{int} = n](x : \mathbf{int}). x * c) \mathbf{as} \alpha)$$

Note that the constraint $(\lambda[c : \mathbf{int}](x : \mathbf{int}). x * c) \in \alpha$ makes no mention of the capture expression n . As formalized in Figure 2, the lambdas in lambda inclusion constraints are of a different syntactic category than lambda expressions and *do not* track capture expressions.

The following example, which references *twice* and *make-mult* as defined in our previous examples, demonstrates a call to a function whose L^{annot} type requires an inclusion constraint²:

$$\mathbf{let\ } f = \mathbf{match\ (get-user-input())} \left\{ \begin{array}{l} 0 \rightarrow \mathbf{make-mult}(5) \\ 1 \rightarrow \mathbf{make-mult}(7) \\ 2 \rightarrow \lambda(x : \mathbf{int}). x + 1 \end{array} \right\} \mathbf{in}$$

$$\mathbf{twice}(f, 20)$$

After running our type inference, this L^{src} program would be transformed to the following well-typed L^{annot} program, which satisfies all relevant inclusion constraints and ensures that f is typed with the same lambda set everywhere.³

$$\mathbf{let\ } f = \mathbf{match\ (get-user-input())} \left\{ \begin{array}{l} 0 \rightarrow \mathbf{make-mult}(\{(\lambda[c](x). x * c), (\lambda(x). x + 1)\})(5) \\ 1 \rightarrow \mathbf{make-mult}(\{(\lambda[c](x). x * c), (\lambda(x). x + 1)\})(7) \\ 2 \rightarrow (\lambda(x). x + 1) \mathbf{as} \{(\lambda[c](x). x * c), (\lambda(x). x + 1)\} \end{array} \right\} \mathbf{in}$$

$$\mathbf{twice}(\{(\lambda[c](x). x * c), (\lambda(x). x + 1)\}(f, 20)$$

Although the calls *make-mult*(5) and *make-mult*(7) return different concrete function values, the L^{annot} type system models them both as simply returning a value with type $\mathbf{int} \xrightarrow{\alpha} \mathbf{int}$ with inclusion constraint $(\lambda[c](x). x * c) \in \alpha$.

Selected key typing rules for L^{annot} , including a treatment of lambda inclusion constraints, are given in Figure 3, and the full typing rules for L^{annot} are given in Appendix C. Expressions are typed in a three-part context consisting of a **def**-binding context Δ , a constraint context Q , and a local binding context Γ . L^{annot} programs are typed in only a **def**-binding context Δ . For a lambda $\ell = \lambda[x : \tau_1](y : \tau_2).(\epsilon_2 : \tau_3)$, we define the notation $\ell[\epsilon_1]$ to denote the lambda-abstraction-with-capture $\ell[\epsilon_1] = \lambda[x : \tau_1 = \epsilon_1](y : \tau_2).(\epsilon_2 : \tau_3)$.

²We match on **get-user-input**() to emphasize LSS’s ability to accommodate dynamic control flow.

³We omit some “ $\dots : \mathbf{int}$ ” type annotations in this example for space.

$$\begin{array}{l}
\boxed{\mathcal{F}(t) = \tau} \quad \mathcal{F}(t_1 \rightarrow t_2) = \mathcal{F}(t_1) \xrightarrow{\alpha} \mathcal{F}(t_2) \quad \text{for } \alpha \text{ fresh} \\
\boxed{\mathcal{F}(\Delta, e) = \epsilon} \quad \mathcal{F}(\Delta, \lambda[x : t_1 = e_1](y : t_2).(e_2 : t_3)) \\
\quad = (\lambda[x : \mathcal{F}(t_1) = \mathcal{F}(\Delta, e_1)](y : \mathcal{F}(t_2)).(\mathcal{F}(\Delta, e_2) : \mathcal{F}(t_3))) \text{ as } \alpha \quad \text{for } \alpha \text{ fresh} \\
\quad \mathcal{F}(\Delta, e_1(e_2)) = (\mathcal{F}(\Delta, e_1) \text{ as } \alpha)(\mathcal{F}(\Delta, e_2)) \\
\mathcal{F}(\Delta, d) = \begin{cases} d\langle\beta_1, \dots, \beta_n\rangle \text{ for } \beta_1, \dots, \beta_n \text{ fresh} & \text{if } (d : \forall \alpha_1, \dots, \alpha_n. Q \Rightarrow \tau) \in \Delta \\ d\langle\rangle & \text{otherwise} \end{cases}
\end{array}$$

Fig. 4. Selected Rules for $\mathcal{F}$.

4.2 Type Inference for L^{annot}

Automatically elaborating well-typed L^{src} programs into well-typed, polymorphic L^{annot} programs is tractable. To do this, it suffices to run a unification-based type inference algorithm [Hindley 1969; Milner 1978] on the body of each definition, augmented with features to handle our lambda inclusion constraints.

We present the details of our type inference procedure, and characterize its correctness with respect to the L^{annot} type system via Theorem 4.1. An Isabelle/HOL formalization of this section, including a proof of Theorem 4.1, is available (see [Artifact Availability](#)).

4.2.1 Unification of Lambda Set Variables. Our type inference proceeds from the observation that the L^{annot} typing rules for expressions, *minus lambda inclusion constraints*, induce only simple equality constraints between L^{annot} types. We call an L^{annot} expression *pre-well-typed* if it is well-typed ignoring lambda inclusion constraints.

The first major component of our type inference is a procedure for elaborating well-typed L^{src} definitions into pre-well-typed L^{annot} definitions. We achieve this by way of unification. Given a well-typed L^{src} expression e , we:

- (1) Annotate e with fresh lambda set variables to obtain ϵ in the expression grammar of L^{annot} .
- (2) Accumulate the set of *type equality constraints* ξ induced by ϵ .
- (3) Determine the set of *lambda set variable equality constraints* ζ implied by ξ .
- (4) Compute the most general lambda set variable unifier ϕ which satisfies ζ .
- (5) Apply the unifier ϕ to obtain a pre-well-typed L^{annot} expression $\epsilon' = \phi\epsilon$.

We formalize step (1) in terms of an imperative procedure $\mathcal{F}$ which lifts L^{src} types and expressions into the grammar of L^{annot} by annotating them with fresh lambda set variables. Selected rules for $\mathcal{F}$ are given in Figure 4, and a complete definition is given in Appendix D. Note that when $\mathcal{F}$ is applied to expressions it takes an additional context parameter Δ , which holds the signatures of prior **def** bindings in the program; this context is necessary in order to determine the number of lambda set parameters with which referenced definitions must be instantiated. Note also that type inference always takes place *inside* the body of some L^{src} **def** binding, and that the *current* enclosing definition will never be present in Δ . When $\mathcal{F}$ encounters a recursive reference to the current definition, it leaves that reference's lambda set parameter list empty, to be filled in later when type inference for the current definition is complete.

We formalize step (2) in terms of a *unification constraint accumulation* relation $\Delta; \Sigma; \Gamma \vdash \epsilon \rightsquigarrow_U \tau; \xi$. The rules for this relation are syntax-directed, and should be understood as defining a procedure with τ and ξ as outputs. Given a **def** context Δ and a local context Γ , the relation $\epsilon \rightsquigarrow_U \tau; \xi$ states that ϵ is inferred to have type τ , and that ϵ induces a set of type equality constraints ξ . The special

$$\begin{aligned}
\zeta \in \text{Type-Equalities} &::= \{\tau_1 \sim \tau'_1, \dots, \tau_n \sim \tau'_n\} & \boxed{\Delta; \Sigma; \Gamma \vdash \epsilon \rightsquigarrow_U \tau; \zeta} \\
\zeta \in \text{Lambda-Set-Var-Equalities} &::= \{\alpha_1 \sim \alpha'_1, \dots, \alpha_n \sim \alpha'_n\} & \boxed{\mathcal{E}(\zeta) = \zeta} \\
\frac{\Delta; \Sigma; \Gamma \vdash \epsilon_1 \rightsquigarrow_U \tau'_1; \xi_1 \quad \Delta; \Sigma; \Gamma \cup \{x : \tau_1, y : \tau_2\} \vdash \epsilon_2 \rightsquigarrow_U \tau'_3; \xi_2 \quad \xi = \xi_1 \cup \xi_2}{\Delta; \Sigma; \Gamma \vdash (\lambda[x : \tau_1 = \epsilon_1](y : \tau_2).(\epsilon_2 : \tau_3)) \text{ as } \alpha \rightsquigarrow_U \tau_2 \xrightarrow{\alpha} \tau_3; \xi \cup \{\tau_1 \sim \tau'_1, \tau_3 \sim \tau'_3\}} & [\text{TIU-ABS}] \\
\frac{\Delta; \Sigma; \Gamma \vdash \epsilon_1 \rightsquigarrow_U \tau_1 \xrightarrow{\alpha_1} \tau_2; \xi_1 \quad \Delta; \Sigma; \Gamma \vdash \epsilon_2 \rightsquigarrow_U \tau_3; \xi_2}{\Delta; \Sigma; \Gamma \vdash (\epsilon_1 \text{ as } \alpha_2)(\epsilon_2) \rightsquigarrow_U \tau_3; \xi_1 \cup \xi_2 \cup \{(\tau_1 \xrightarrow{\alpha_1} \tau_2) \sim (\tau_3 \xrightarrow{\alpha_2} \tau_2)\}} & [\text{TIU-APP}] \\
\frac{(d : \forall \bar{\alpha}. Q \Rightarrow \tau) \in \Delta}{\Delta; \Sigma; \Gamma \vdash d\langle \bar{\beta} \rangle \rightsquigarrow_U \tau[\bar{\alpha} \mapsto \bar{\beta}]; \emptyset} & [\text{TIU-DEF-REF}] \quad \frac{}{\Delta; (d : \tau); \Gamma \vdash d \rightsquigarrow_U \tau; \emptyset} & [\text{TIU-SELF-REF}]
\end{aligned}$$

Fig. 5. Selected Rules for Unification Constraint Accumulation.

$$\begin{aligned}
\boxed{Q(\Delta, \epsilon) = Q} \quad & Q(\Delta, (\lambda[x : \tau_1 = \epsilon_1](y : \tau_2).(\epsilon_2 : \tau_3)) \text{ as } \alpha) \\
&= Q(\Delta, \epsilon_1) \cup Q(\Delta, \epsilon_2) \cup \{(\lambda[x : \tau_1](y : \tau_2).(\epsilon_2 : \tau_3)) \in \alpha\} \\
Q(\Delta, d\langle \bar{\sigma} \rangle) &= \begin{cases} Q[\bar{\alpha} \mapsto \bar{\sigma}] & \text{if } (d : \forall \bar{\alpha}. Q \Rightarrow \tau) \in \Delta \\ \emptyset & \text{otherwise} \end{cases}
\end{aligned}$$

Fig. 6. Selected Rules for Lambda Inclusion Constraint Accumulation.

context parameter Σ tracks a provisional L^{annot} type for the *current enclosing definition*, which we use to ensure that recursive references to the current definition have the same type as the current definition's body. Tracking the inferred type of the current definition in this way ensures by construction that our elaboration procedure never introduces polymorphic recursion. Selected rules for the $\rightsquigarrow_U$ relation are given in Figure 5, and the complete rules are given in Appendix D.

Note that $\rightsquigarrow_U$ defines a *partial* function, and can fail in cases where inferred types do not match the structure expected by any of its defining rules (for example, if ϵ_1 fails to yield a type of the form $\tau_1 \xrightarrow{\alpha_1} \tau_2$ in TIU-APP). However, when operating on input programs which were well-typed under the L^{src} type system, such structural incompatibilities cannot occur, and $\rightsquigarrow_U$ always succeeds.

We formalize step (3) in terms of a procedure $\mathcal{E}$ which recursively traverses the types in each type equality constraint $\tau_1 \sim \tau_2$ to obtain a set of implied lambda set equalities. The form of $\mathcal{E}$ is given in Figure 5, and a complete definition is given in Appendix D. Note that in order to ensure that $\mathcal{E}$ always emits constraints which we can solve by unification, we require that each implied lambda set equality take the form $\alpha_1 \sim \alpha_2$ where α_1, α_2 are lambda set *variables*, not concrete lambda sets. To guarantee this, we proceed from the observation that lambda set annotations encountered during type inference can come from two places: they may be introduced by $\mathcal{F}$, or they may be retrieved from Δ . In $\mathcal{F}$, we ensure by construction that all new lambda set annotations are fresh variables. In Δ , we maintain an inductive invariant throughout type inference that all inferred **def** signatures are *fully flexible*, in the sense that all their lambda set annotations take the form of universally quantified lambda set variables.

Together with steps (4) and (5), which are standard, this completes the unification component of our algorithm.

$$\begin{aligned}
\text{lsv}(\sigma) &= \{\alpha \mid \alpha \text{ appears in } \sigma\} & \boxed{\mathcal{S}(Q, \alpha) = \sigma} & \quad \boxed{\mathcal{I}(\{\alpha_1, \dots, \alpha_n\}, Q, \epsilon) = (Q, \epsilon)} \\
\mathcal{S}(Q, \alpha) &= \begin{cases} \mu a. \{\ell[\alpha \mapsto a] \mid (\ell \in \alpha) \in Q\} \text{ for a fresh } a & \text{if } \alpha \in \text{lsv}(\{\ell \mid (\ell \in \alpha) \in Q\}) \\ \{\ell \mid (\ell \in \alpha) \in Q\} & \text{otherwise} \end{cases} \\
\mathcal{I}(\emptyset, Q, \epsilon) &= (Q, \epsilon) \\
\mathcal{I}(\{\alpha_1, \dots, \alpha_n\}, Q, \epsilon) &= \mathcal{I}(\{\alpha_2, \dots, \alpha_n\}, Q[\alpha_1 \mapsto \sigma], \epsilon[\alpha_1 \mapsto \sigma]) \quad \text{where } \sigma = \mathcal{S}(Q, \alpha_1)
\end{aligned}$$

Fig. 7. The $\mathcal{S}$ and $\mathcal{I}$ functions.

4.2.2 Inclusion Constraints and Internalization. Although our unification algorithm suffices to lift each **def** binding in an L^{src} program to a pre-well-typed L^{annot} **def** binding, additional measures specific to our setting are necessary to ensure that the L^{annot} program generated by type inference also satisfies the lambda inclusion constraints imposed by the AT-ABS and AT-DEF-REF rules.

To ensure that all lambda inclusion constraints are satisfied, we first define a procedure $\mathcal{Q}$ which returns the set of inclusion constraints induced by an L^{annot} expression. The procedure $\mathcal{Q}$ is parameterized by a **def** context Δ , because the AT-DEF-REF typing rule (given in Figure 3) allows references to definitions to impose inclusion constraints. Selected rules for $\mathcal{Q}$ are given in Figure 6.

Our type inference then uses the constraints returned by $\mathcal{Q}$ in two ways:

- Lambda set variables appearing free in the *type signature* of the current definition are promoted to universally quantified lambda set parameters, and any inclusion constraints on these variables are added to the constraint set for the current definition.
- Variables *not* appearing in the type signature are “internalized,” meaning that they are replaced with concrete lambda sets sufficient to satisfy all their inclusion constraints.

We formalize this “internalization” procedure in Figure 7, in terms of two functions $\mathcal{S}$ and $\mathcal{I}$. The function $\mathcal{S}$ gives a minimal concrete solution for a lambda set variable α subject to a set of inclusion constraints Q . The function $\mathcal{I}$ “internalizes” a sequence of lambda set variables $\{\alpha_1, \dots, \alpha_n\}$ in the body ϵ of a definition, replacing those variables by their minimal concrete solutions given Q .

During internalization, we note that a difficulty arises when attempting to generate a solution for a lambda set variable which *occurs free in its own constraints*. Consider the problem of generating a lambda set γ satisfying a single constraint of the form:

$$(\lambda[f : \text{int} \xrightarrow{\gamma} \text{int}](x : \text{int}). (f \text{ as } \gamma)(x) + 1) \in \gamma$$

As noted by [Hovgaard \[2018\]](#), constraints like the above can arise in the presence of recursive or iterative control flow. To represent concrete lambda sets satisfying such recursive constraints without generating infinite-size lambda set expressions, we introduce a lambda-set-level fixed point operator μ . Using μ , we can express a concrete solution for γ satisfying the original constraint:

$$\gamma = \mu a. (\lambda[f : \text{int} \xrightarrow{a} \text{int}](x : \text{int}). (f \text{ as } a)(x) + 1)$$

The typing rule for μ lambda sets is given as part of Figure 3, and the rule which introduces μ lambda sets during type inference is given as part of the definition of $\mathcal{S}$ in Figure 7.

4.2.3 The End-to-End Type Inference Procedure. Putting all the components of our type inference together, we obtain a procedure $\mathcal{D}$, presented in Figure 8, which applies our full type inference technique to a single definition. Given a well-typed L^{src} definition **def** $d : t = e$ and a context Δ holding inferred L^{annot} signatures for all prior definitions in the program, the procedure $\mathcal{D}(\Delta, d, t, e)$ returns a tuple $(\{\bar{\alpha}\}, Q, \tau'', \epsilon''')$ containing the lambda set parameters $\bar{\alpha}$, inclusion constraint set

$$\begin{aligned}
\mathcal{D}(\Delta, d, t, e) &= (\{\bar{\alpha}\}, Q, \tau'', \epsilon''') \text{ where} \\
\tau &= \mathcal{F}(t) \quad \epsilon = \mathcal{F}(\Delta, e) \quad \Delta; (d : \tau); \emptyset \vdash \epsilon \rightsquigarrow_U \tau'; \xi \quad \phi = \text{mgu}(\mathcal{E}(\xi \cup \{\tau \sim \tau'\})) \quad \tau'' = \phi\tau' \\
\epsilon' &= \phi\epsilon \quad \{\bar{\alpha}\} = \text{lsv}(\tau'') \quad \epsilon'' = \epsilon'[d\langle \rangle \mapsto d\langle \bar{\alpha} \rangle] \quad (Q, \epsilon''') = \mathcal{I}((\text{lsv}(\epsilon'') \setminus \{\bar{\alpha}\}), \mathcal{Q}(\Delta, \epsilon''), \epsilon'') \\
\mathcal{A}(\Delta, d) &= d\langle \rangle \\
\mathcal{A}(\Delta, (\mathbf{def} \, d : t = e \, \mathbf{in} \, p)) &= (\mathbf{def} \, d\langle \bar{\alpha} \rangle : (Q \Rightarrow \tau) = \epsilon \, \mathbf{in} \, \pi) \text{ where} \\
(\{\bar{\alpha}\}, Q, \tau, \epsilon) &= \mathcal{D}(\Delta, d, t, e) \quad \pi = \mathcal{A}(\Delta \cup \{d : \forall \bar{\alpha}. Q \Rightarrow \tau\}, p)
\end{aligned}$$

Fig. 8. Type Inference

Q , type signature τ'' , and body ϵ''' of a corresponding well-typed L^{annot} definition. The procedure $\mathcal{D}$ first runs the unification algorithm described in Section 4.2.1 to obtain a most general unifier (“mgu”) ϕ which it applies to the definition, then accumulates inclusion constraints and performs internalization as described in Section 4.2.2. The free variables $\{\bar{\alpha}\}$ of the inferred type signature τ'' become the universally quantified lambda set parameters of the definition, and we apply the substitution $d\langle \rangle \mapsto d\langle \bar{\alpha} \rangle$ to fix any temporary placeholder references to the current definition which were introduced in the $\mathcal{F}$ procedure.

The procedure $\mathcal{A}$, also presented in Figure 8, defines our type inference technique at the level of whole L^{src} programs, processing **def** bindings one-by-one in dependency order. We express our central theoretical result for type inference in terms of $\mathcal{A}$:

THEOREM 4.1 (SOUNDNESS AND COMPLETENESS OF TYPE INFERENCE WITH RESPECT TO THE L^{ANNOT} TYPE SYSTEM). *Let p be an L^{src} program, and let $\emptyset \vdash p : t$ where t contains no occurrences of the function type constructor $\cdot \rightarrow \cdot$. Then computing $\pi = \mathcal{A}(\Delta, p)$ always succeeds, and gives π such that:*

- We have $\lfloor \pi \rfloor = p$.
- We have $\emptyset \vdash \pi : \tau$ where $\lfloor \tau \rfloor = t$.
- The program π contains no instances of polymorphic recursion.

5 LOWERING L^{annot} PROGRAMS TO FIRST-ORDER

The lambda set annotations in the L^{annot} language are designed to track all information necessary to defunctionalize a higher-order program to an equivalent first-order form. Although a polymorphic L^{annot} program cannot be *directly* defunctionalized, it can be defunctionalized after *specializing* **def** bindings with respect to concrete lambda set parameters, producing a monomorphic L^{annot} program annotated entirely by concrete lambda sets. In this section, we first discuss our specialization procedure and then the lowering procedure for monomorphic L^{annot} programs.

5.1 Specialization of L^{annot} Programs

The procedure for specializing definitions in L^{annot} with respect to lambda sets is straightforward, and proceeds similarly to the type monomorphization of languages such as C++ and Rust.

At the end of Section 4.1, we introduced the L^{src} and inferred L^{annot} code for an example program which matches on user input, calling the function *twice* with either a concrete lambda or the result of a call to higher-order function *make-mult*. The L^{annot} code calls both functions, where they appear, with lambda set parameter $\sigma = \{(\lambda[c](x). x * c), (\lambda(x). x + 1)\}$. Hence, when we specialize, we get a program with a single specialization of *twice* and of *make-mult*, each with respect to σ :

def *make-mult*_{spec} $\langle \rangle : \mathbf{int} \rightarrow (\mathbf{int} \xrightarrow{\{(\lambda[c](x). x * c), (\lambda(x). x + 1)\}} \mathbf{int}) =$

```

λ(n). ((λ[c = n](x). x * c) as {(λ[c](x). x * c), (λ(x). x + 1)}) in
def twicespec⟨⟩ : ((int  $\xrightarrow{\{(\lambda[c](x). x * c), (\lambda(x). x + 1)\}}$  int), int) → int =
  λ(f, x). (f as {(λ[c](x). x * c), (λ(x). x + 1)})((f as {(λ[c](x). x * c), (λ(x). x + 1)})(x))

let f = match (get-user-input())  $\left\{ \begin{array}{l} 0 \rightarrow \text{make-mult}_{\text{spec}}\langle\rangle(5) \\ 1 \rightarrow \text{make-mult}_{\text{spec}}\langle\rangle(7) \\ 2 \rightarrow (\lambda(x). x + 1) \text{ as } \{(\lambda[c](x). x * c), (\lambda(x). x + 1)\} \end{array} \right\}$  in
twicespec⟨⟩(f, 20)

```

We present rules formalizing our specialization procedure in Figure 9. The algorithm works iteratively, starting from the entry point at the end of the program and traversing the program’s list of **def** bindings in reverse dependency order—the “outer specialization loop” in Figure 9. As we step backwards through the list of **def** bindings, we replace each (potentially polymorphic) binding by a set of zero or more monomorphic specialized copies, and replace references to the original binding elsewhere in the program with references to these new specialized copies—the “inner specialization loop” in Figure 9. For each binding, we determine the set of specializations to generate for it by inspecting the references to it which exist in the parts of the program which have already been specialized. Note that the intermediate programs produced during specialization might reference undefined **def** bindings.

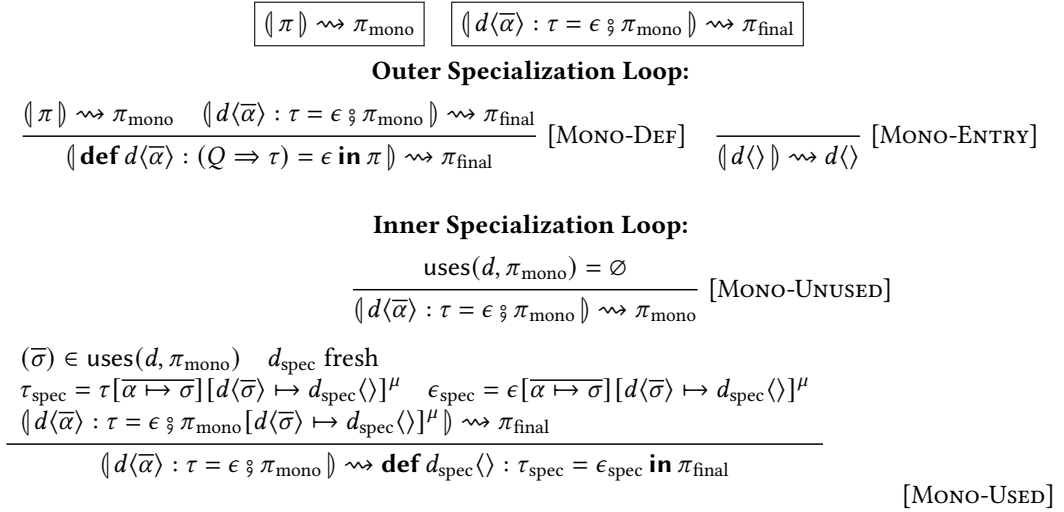


Fig. 9. The Monomorphization Procedure for L^{annot}

We can formally characterize the behavior of our specialization procedure in terms of the following soundness property:

THEOREM 5.1 (SOUNDNESS OF SPECIALIZATION). *Suppose we have $\Delta \vdash \pi : \tau$ and that π contains no instances of polymorphic recursion. Then:*

- **Specialization Terminates:** *There exists π_{mono} such that $\langle \pi \rangle \rightsquigarrow \pi_{\text{mono}}$.*
- **Specialization Monomorphizes:** *The program π_{mono} is monomorphic.*
- **Specialization Preserves Types:** *We have $\Delta \vdash \pi_{\text{mono}} : \tau$.*
- **Specialization Preserves Semantics:** *We have $\Delta \vdash \lfloor \pi_{\text{mono}} \rfloor \Downarrow v$ if and only if $\Delta \vdash \lfloor \pi \rfloor \Downarrow v$.*

These properties follow as a corollary from the soundness of the inner loop:

LEMMA 5.2 (SOUNDNESS OF SPECIALIZATION INNER LOOP). *Let π be given by*

$$\pi = (\mathbf{def} \, d\langle\bar{\alpha}\rangle : (Q \Rightarrow \tau_d) = \epsilon_d \mathbf{in} \, \pi_{\text{mono}})$$

Suppose that π contains no instances of polymorphic recursion, that π_{mono} is monomorphic, and that $\Delta \vdash \pi : \tau$. Then:

- **Inner Loop Terminates:** *There exists π_{final} such that $\langle d\langle\bar{\alpha}\rangle : \tau_d = \epsilon_d \circ \pi_{\text{mono}} \rangle \rightsquigarrow \pi_{\text{final}}$.*
- **Inner Loop Monomorphizes:** *The program π_{final} is monomorphic.*
- **Inner Loop Preserves Types:** *We have $\Delta \vdash \pi_{\text{final}} : \tau$.*
- **Inner Loop Preserves Semantics:** *We have $\Delta \vdash \lfloor \pi_{\text{final}} \rfloor \Downarrow v$ if and only if $\Delta \vdash \lfloor \pi \rfloor \Downarrow v$.*

Recall that our type inference procedure never introduces instances of polymorphic recursion, so the conditions of Theorem 5.1 hold for any L^{annot} program produced by our type inference.

In the remainder of this section we explain how the inner loop procedure upholds the properties stated in Lemma 5.2.

To understand the inner loop rules MONO-UNUSED and MONO-USED, first note that the notation ‘uses(d, π_{mono})’ used in these rules denotes the set of all lambda set parameter lists with which d is instantiated in π_{mono} . This justifies the MONO-UNUSED rule; if a definition d is not mentioned anywhere in a program π_{mono} , then removing d from the context of π_{mono} has no effect on its typing or semantics.

We now turn our attention to the MONO-USED rule. In MONO-USED, we produce a version of the definition $d\langle\bar{\alpha}\rangle : (Q \Rightarrow \tau) = \epsilon$ specialized to the lambda set parameters $\bar{\sigma}$, and replace any references to $d\langle\bar{\sigma}\rangle$ which occur in π_{mono} with references to its new specialization $d_{\text{spec}}\langle\cdot\rangle$. To preserve our typing rules, it turns out to be necessary to perform this replacement $d\langle\bar{\sigma}\rangle \mapsto d_{\text{spec}}\langle\cdot\rangle$ using a nonstandard substitution strategy which we refer to as μ -aware substitution. We notate our μ -aware substitution operator as $(\cdot)[\cdot \mapsto \cdot]^\mu$, and explain its definition and motivation in more detail in Section 5.1.1. For the purposes of understanding the rule MONO-USED, it is sufficient to note that both lambda set substitution and μ -aware substitution preserve expression typing. This is formalized in lemmas 5.3 and 5.5. We state Lemma 5.3 here:

LEMMA 5.3 (LAMBDA SET VARIABLE SUBSTITUTION PRESERVES EXPRESSION TYPING). *Let $Q, Q', \bar{\alpha}, \bar{\sigma}$ satisfy $Q' \vdash Q[\bar{\alpha} \mapsto \bar{\sigma}]$. Then for any $\Delta, \Gamma, \epsilon, \tau$ we have that*

$$\Delta ; Q ; \Gamma \vdash \epsilon : \tau \implies \Delta ; Q' ; \Gamma[\bar{\alpha} \mapsto \bar{\sigma}] \vdash \epsilon[\bar{\alpha} \mapsto \bar{\sigma}] : \tau[\bar{\alpha} \mapsto \bar{\sigma}]$$

Lemma 5.3 follows directly from the typing rules for L^{annot} . We defer the formal statement of Lemma 5.5 to Section 5.1.1.

Using Lemma 5.3 and Lemma 5.5, it follows that each specialized definition d_{spec} produced by MONO-USED is well-typed and monomorphic. In particular, suppose that π_{mono} is monomorphic and that we have

$$\Delta \vdash (\mathbf{def} \, d\langle\bar{\alpha}\rangle : (Q \Rightarrow \tau_d) = \epsilon_d \mathbf{in} \, \pi_{\text{mono}}) : \tau$$

Then let $\bar{\sigma} \in \text{uses}(d, \pi_{\text{mono}})$. From the assumption that π_{mono} is monomorphic, we know $\bar{\sigma}$ must be monomorphic. Moreover, from the fact that π_{mono} is well-typed in a context which includes d , we know that we must have $\emptyset \vdash Q[\bar{\alpha} \mapsto \bar{\sigma}]$. Letting $\tau_{\text{spec}}, \epsilon_{\text{spec}}$ be given by

$$\tau_{\text{spec}} = \tau_d[\bar{\alpha} \mapsto \bar{\sigma}][d\langle\bar{\sigma}\rangle \mapsto d_{\text{spec}}\langle\cdot\rangle]^\mu \quad \epsilon_{\text{spec}} = \epsilon_d[\bar{\alpha} \mapsto \bar{\sigma}][d\langle\bar{\sigma}\rangle \mapsto d_{\text{spec}}\langle\cdot\rangle]^\mu$$

we then have by lemmas 5.3 and 5.5 that the specialized definition $d_{\text{spec}}\langle\cdot\rangle : \tau_{\text{spec}} = \epsilon_{\text{spec}}$ is monomorphic and well-typed in Δ .

$$\begin{aligned}
& \psi \in \text{Def-Subst} ::= (d_1 \langle \bar{\sigma} \rangle \mapsto d_2 \langle \rangle) \\
& \boxed{\mathcal{M}(\epsilon, \epsilon', \psi) = \epsilon''} \quad \mathcal{M}(d \langle \bar{\sigma} \rangle, d \langle \bar{\sigma}' \rangle, d_1 \langle \bar{\sigma}_1 \rangle \mapsto d_2 \langle \rangle) = \begin{cases} d_2 \langle \rangle & \text{if } d = d_1, \bar{\sigma}' = \bar{\sigma}_1 \\ \mathcal{M}(d \langle \bar{\sigma} \rangle, d \langle \bar{\sigma}' \rangle, \psi) & \text{otherwise} \end{cases} \\
& \boxed{\mathcal{M}(\sigma, \sigma', \psi) = \sigma''} \quad \mathcal{M}(\{\ell_1, \dots, \ell_n\}, \{\ell'_1, \dots, \ell'_n\}, \psi) = \{\mathcal{M}(\ell_1, \ell'_1, \psi), \dots, \mathcal{M}(\ell_n, \ell'_n, \psi)\} \\
& \mathcal{M}(\alpha, \alpha, \psi) = \alpha \\
& \mathcal{M}(\mu a. \sigma, \mu a. \sigma', \psi) = \mu a. (\mathcal{M}(\sigma, \sigma' [a \mapsto \mu a. \sigma'], \psi)) \\
& \mathcal{M}(a, \sigma', \psi) = a \\
& \boxed{\epsilon[\psi]^\mu = \epsilon'} \quad \epsilon[\psi]^\mu = \mathcal{M}(\epsilon, \epsilon, \psi) \quad \boxed{\sigma[\psi]^\mu = \sigma'} \quad \sigma[\psi]^\mu = \mathcal{M}(\sigma, \sigma, \psi)
\end{aligned}$$

Fig. 10. Selected Rules for μ -Aware Substitution.

Lemma 5.2 then follows by induction on the number of remaining occurrences of d in π_{mono} , which is monotonically decreasing on each MONO-USED step thanks to the assumption that the original polymorphic program contains no instances of polymorphic recursion.

5.1.1 μ -Aware Substitution. On each step of our specialization procedure, we need to replace references to a definition d with references to a specialized definition d_{spec} everywhere d appears instantiated with parameters $\bar{\sigma}$. It turns out that to perform this substitution in a manner which preserves types, it is not quite sufficient to simply replace every syntactic occurrence of $d \langle \bar{\sigma} \rangle$ by a reference to the new specialized definition $d_{\text{spec}} \langle \rangle$. Instead, it is necessary to also replace expressions which *would* match $d \langle \bar{\sigma} \rangle$ if μ binders were unfolded. To accommodate this, we define a special *μ -aware substitution* operator, which we describe in this section.

To see why naïve syntactic substitution is not sufficient, first note that the following judgment is valid in our type system:

$$\emptyset \vdash (\lambda x. d \langle \mu a. \{\lambda x. d \langle a \rangle\} \rangle) \in \mu a. \{\lambda x. d \langle a \rangle\}$$

This is because the μ lambda set on the right hand side can be *unfolded* as:

$$\{\lambda x. d \langle a \rangle\} [a \mapsto \mu a. \{\lambda x. d \langle a \rangle\}] = \{\lambda x. d \langle \mu a. \{\lambda x. d \langle a \rangle\} \rangle\}$$

This gives a lambda set expression containing the lambda on the left hand side. If we were to apply the naïve substitution $d \langle \mu a. \{\lambda x. d \langle a \rangle\} \rangle \mapsto d_{\text{spec}} \langle \rangle$ to both sides of this judgment, we would obtain the judgment:

$$\emptyset \vdash (\lambda x. d_{\text{spec}} \langle \rangle) \in \mu a. \{\lambda x. d \langle a \rangle\}$$

This judgment does *not* hold under our type system. To obtain a valid judgment, we would need to also replace the expression $d \langle a \rangle$ in the right hand side μ lambda set with $d_{\text{spec}} \langle \rangle$, so that the transformed right hand side still unfolds to a lambda set containing the transformed left hand side:

$$\emptyset \vdash (\lambda x. d_{\text{spec}} \langle \rangle) \in \mu a. \{\lambda x. d_{\text{spec}} \langle a \rangle\}$$

To achieve this desired substitution behavior, we introduce a *μ -aware substitution* operator $(\cdot)[\cdot \mapsto \cdot]^\mu$. Selected rules are given in Figure 10 and the full definition is given in appendix F. The substitution procedure is implemented in terms of a helper function $\mathcal{M}$. Each call to $\mathcal{M}(\epsilon, \epsilon', \psi)$ recursively tracks both the current term ϵ into which to apply the substitution ψ , as well as an “unfolded version” ϵ' of the current term which corresponds to what we would obtain if all enclosing μ binders in the context surrounding ϵ were to be unfolded. When we reach a term which is a

reference to a definition, we consult the unfolded version of the term to determine if its lambda set arguments match those appearing in the left hand side of the substitution.

Unlike naïve syntactic substitution, μ -aware substitution satisfies the following:

LEMMA 5.4 (μ -AWARE DEF SUBSTITUTION PRESERVES CONSTRAINT ENTAILMENT). *Let ψ stand for the def substitution $\psi = (d_1 \langle \bar{\sigma} \rangle \mapsto d_2 \langle \rangle)$. Then, for any Q, ℓ, σ we have:*

$$Q \vdash \ell \in \sigma \implies Q[\psi]^\mu \vdash \ell[\psi]^\mu \in \sigma[\psi]^\mu$$

This result follows from an induction on the constraint judgment, taking advantage of the rules determining how $\mathcal{M}(\cdot, \cdot, \cdot)$ acts on μ lambda sets. Lemma 5.4 accounts for the core case of a much stronger lemma, which states that using μ -aware substitution to replace occurrences of a polymorphic definition d with a compatible specialization d_{spec} preserves typing in L^{annot} :

LEMMA 5.5 (μ -AWARE DEF SUBSTITUTION PRESERVES TYPING). *Let the L^{annot} definition typing context Δ decompose as*

$$\Delta = \Delta_{\text{other}} \cup \{d : \forall \bar{\alpha}. Q_d \implies \tau_d\}$$

and let $\bar{\sigma}$ be a sequence of monomorphic lambda sets of the same length as $\bar{\alpha}$. Let ψ denote the def substitution $\psi = (d \langle \bar{\sigma} \rangle \mapsto d_{\text{spec}})$, let $\tau_{\text{spec}} = \tau_d[\bar{\alpha} \mapsto \bar{\sigma}][\psi]^\mu$, and let Δ_{spec} be given by

$$\Delta_{\text{spec}} = \Delta_{\text{other}}[\psi]^\mu \cup \{(d : \forall \bar{\alpha}. Q_d \implies \tau_d), (d_{\text{spec}} : \tau_{\text{spec}})\}$$

Then:

- **Def Substitution Preserves Expression Typing:** *For any $Q, \Gamma, \epsilon, \tau$, we have*

$$\Delta ; Q ; \Gamma \vdash \epsilon : \tau \implies \Delta_{\text{spec}} ; Q[\psi]^\mu ; \Gamma[\psi]^\mu \vdash \epsilon[\psi]^\mu : \tau[\psi]^\mu$$

- **Def Substitution Preserves Program Typing:** *For any π , we have*

$$\Delta \vdash \pi : \tau \implies \Delta_{\text{spec}} \vdash \pi[\psi]^\mu : \tau[\psi]^\mu$$

Note that the statement of Lemma 5.5 is, beyond parsing out definitions of interest from Δ , a straightforward substitution lemma which doesn't even *mention* μ . This demonstrates that μ doesn't complicate our formal results.

5.2 Lowering of Monomorphic L^{annot} Programs To First-Order

Lambda set annotations in a monomorphic L^{annot} program can be used to guide a lowering process which transforms the L^{annot} program to a first-order representation. The target of this lowering process is a simple first-order language L^{FO} and the basic strategy underlying our lowering procedure is the same as that used in existing defunctionalization techniques such as [Cejtin et al. 2000]. Namely, function types are lowered to sum types where each variant of the sum represents the environment of a different possible lambda which might inhabit the function type. Lambda abstractions are then lowered to injections into these sum types, and function applications are lowered to matches on these sum types. The following is the lowered version of our specialized *make-mult* code from Section 5.1, which demonstrates our lowering strategy in the presence of non-trivial lambda captures:

$$\begin{aligned} \text{let-funs } \text{twice}_{\text{FO}}(f : \text{int} + \text{unit}, x : \text{int}) : \text{int} &= \text{match } f \left\{ \begin{array}{l} \text{inj}_{\text{int}+\text{unit}}^1(c) \rightarrow \text{lam}_1(c, x) \\ \text{inj}_{\text{int}+\text{unit}}^2(()) \rightarrow \text{lam}_2(x) \end{array} \right\} \\ \text{make-mult}_{\text{FO}}(n : \text{int}) : \text{int} + \text{unit} &= \text{inj}_{\text{int}+\text{unit}}^1(n) \\ \text{lam}_1(c : \text{int}, x : \text{int}) : \text{int} &= x * c \\ \text{lam}_2(x : \text{int}) : \text{int} &= x + 1 \quad \text{in} \end{aligned}$$

$$\text{let } f = \text{match } (\text{get-user-input}()) \left\{ \begin{array}{l} 0 \rightarrow \text{make-mult}_{\text{FO}}(5) \\ 1 \rightarrow \text{make-mult}_{\text{FO}}(7) \\ 2 \rightarrow \text{inj}_{\text{int+unit}}^2(()) \end{array} \right\} \text{ in} \\ \text{twice}_{\text{FO}}(f, 20)$$

While very similar to the translation strategy used in prior work on defunctionalization, for completeness we provide Appendix G, which defines and provides semantics for L^{FO} , and formalizes our lowering process in terms of a partial function $\llbracket \cdot \rrbracket_{\text{global}}$ which takes L^{annot} programs to L^{FO} programs. This function enjoys a straightforward semantics preservation property:

THEOREM 5.6 (GLOBAL LOWERING PRESERVES SEMANTICS). *Let π be a monomorphic L^{annot} program such that $\emptyset \vdash \pi : \tau$. Then we have $\emptyset \vdash \llbracket \pi \rrbracket_{\text{global}} \Downarrow v$ if and only if $\emptyset \vdash \lfloor \pi \rfloor \Downarrow v$.*

Composing this result with Theorem 4.1 and 5.1, we obtain the following result showing that LSS preserves semantics end-to-end:

THEOREM 5.7 (LSS PRESERVES SEMANTICS). *Let p be an L^{src} program such that $\emptyset \vdash p : t$ where t contains no occurrences of the function type constructor $\cdot \rightarrow \cdot$. Then:*

- **Elaboration to L^{annot} Succeeds:** *We have π such that $\pi = \mathcal{A}(\emptyset, p)$.*
- **Specialization Succeeds:** *We have π_{mono} such that $\lfloor \pi \rfloor \rightsquigarrow \pi_{\text{mono}}$.*
- **Lowering Succeeds:** *We have $\mathfrak{p}$ such that $\mathfrak{p} = \llbracket \pi_{\text{mono}} \rrbracket_{\text{global}}$.*
- **Semantics are Preserved:** *We have $\emptyset \vdash \mathfrak{p} \Downarrow v$ if and only if $\emptyset \vdash p \Downarrow v$.*

6 IMPLEMENTATION CONSIDERATIONS

Although the formal rules we present for our defunctionalization algorithm are *semantically correct*, a naïve translation of these rules into code is not sufficient to obtain an efficient implementation. From our experience implementing LSS, in this section we briefly describe the implementation details which allow our algorithm to be implemented efficiently in practice.

6.1 Structure Sharing for Lambda Terms

In our formalism, elements of lambda sets are represented as *lambda expressions* of the form

$$\lambda[x : \tau_1](y : \tau_2). \epsilon : \tau_3$$

Recall that the sub-terms τ_1, τ_2, τ_3 and ϵ can recursively contain additional lambda set annotations, thus creating scenarios in which lambda sets will syntactically mention other lambda sets. This is in fact an essential feature of our formalism, as it allows our type system to distinguish between lambdas which differ only by the content of other lambdas on which they operate.

In practice however, lambda sets contain *common subterms* which frequently appear in many different locations. This is a natural consequence of our use of lambda sets as a flow analysis tool; every function call or lambda abstraction to which some lambda set σ flows will be annotated with a distinct lambda set, each of which includes σ . This appearance of a common term on all paths yields an exponentially-sized representation of lambda sets in realistic programs.

Consider, for example, an L^{src} program fragment of the form

```
let f1 = λ(x : int). x in
let f2 = λ[f1 : int → int](x : int). f1(x) + f1(x) in
let f3 = λ[f2 : int → int](x : int). f2(x) + f2(x) in ...
```

In this program, each newly-defined lambda requires a lambda set annotation which includes *all* lambdas previously defined in the program. In particular, the L^{annot} syntax tree for the function f_n

contains the entire body of f_{n-1} inlined multiple times and the size of the L^{annot} syntax tree for f_n is exponential in n .

Thankfully, this exponential blowup is not an essential limitation of our algorithm and is only an artifact of the absence of any binding form for lambda set variables. If we were to simply bind common lambda sets to a name, allowing references to that name to appear in later lambda sets, L^{annot} programs like the above can be easily represented in linear space. Because typical L^{annot} programs equipped with named lambda sets admit representations exponentially smaller than their inlined representations, it is essential to hoist common lambda set subterms to into named definitions. In our implementation, we do this by using a memoization cache.

6.2 Delayed Variable Substitution

Even with common lambda set subterms eliminated, an otherwise naïve implementation of the rules in our formalism would result in an algorithm which needlessly runs in super-linear time on realistic programs. In particular, the type inference and specialization phases of our algorithm both rely heavily on performing variable substitutions on L^{annot} terms, and as demonstrated by the example in Section 6.1, the *depth* of such terms can be realistically expected to be linear in the size of the program. As a result, performing all variable substitutions *eagerly* would cause both type inference and specialization to run in at least quadratic time for such programs.

Again, thankfully, this quadratic behavior is not an essential feature; we can represent lambda sets using a data structure which *delays* variable substitutions until specialization. The idea is to introduce explicit substitutions of lambda term:

$$\ell \in \text{Lambda-Term} ::= (\lambda[x_1 : \tau_1](x_2 : \tau_2).e : \tau_3) \mid \text{Subst}(\ell, [\overline{\alpha \mapsto \sigma}])$$

With this new representation, applying a substitution to a lambda term during type inference takes constant time. Evaluation of the stack of substitutions can be done in linear time, avoiding the need for quadratic, eager substitution application. Crucially, these deferred substitutions need not be forced during type inference. We do note however that our lambda sets may now contain multiple distinct substitution expressions which ultimately evaluate to the same lambda term. This is not a problem, as we already evaluate substitutions before checking lambda sets for equality, cardinality, or inclusion.

6.3 Handling Mutually Recursive Functions

In L^{src} , definitions—as opposed to lambda abstractions—are the unit of specialization. An important property of L^{src} is that an L^{src} definition can only reference itself or previous definitions. This means that the definition dependency graph of an L^{src} program is topologically ordered. Our type inference algorithm requires this property because type inference on a definition d is a function of the type signatures of each of the definitions upon which d depends.

In many programming languages, the function dependency graph for a source program does not necessarily admit a topological order. Nests of mutually-recursive functions may form a strongly-connected component (SCC) in the function dependency graph. Thus, implementations should use SCCs, rather than individual functions, as the unit of specialization. In L^{src} , this means that every SCC in the function dependency graph is converted to an L^{src} **def**, and every function within that component can become a let-bound lambda in that **def**'s body.

7 EVALUATION

When evaluating LSS, we sought to answer three primary research questions:

- (1) Does LSS improve performance over monovariant defunctionalization?
- (2) Does LSS improve performance over non-defunctionalizing compilation?

(3) Does LSS's specialization significantly increase binary size?

To answer these questions, we applied LSS in the setting of three compilers: **Morphic** [Brandon et al. 2023a], **MLton** [Weeks 2006], and **OCaml** [Leroy et al. 2022]. For each of these compilers, we ran a consistent suite of six higher-order benchmark programs. We compiled each program once in a baseline configuration without LSS, and again using LSS as a pre-processing step. We then measured the relative change in each program's run time and binary size after applying LSS.

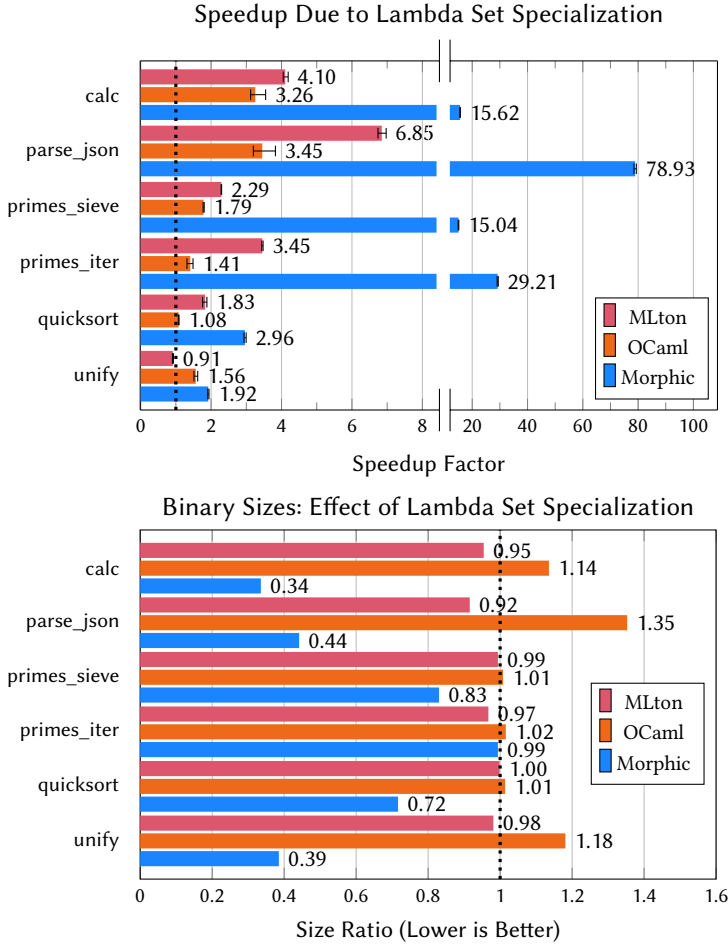


Fig. 11. (Above) Effect of LSS on runtime performance in MLton, OCaml, and Morphic. Numbers reported are the average of 10 runs; error bars represent a ratio of the slowest post-LSS to the fastest pre-LSS run at their minimum, and the fastest post-LSS to the slowest pre-LSS at their maximum. All benchmarks are translated from Morphic, and express speedups obtained after applying Morphic's LSS pass vs a baseline without LSS. (Below) relative binary sizes of compiled benchmarks; numbers below 1 show binaries that shrunk after LSS, while numbers above 1 show binaries which grew. All experiments were conducted on a system with Intel i7-8850 CPU and 32GB Ram. We compiled our benchmarks using MLton version 20210117, and OCaml version 4.14 with `flambda` enabled.

7.1 Experimental Configuration for Morphic, MLton, and OCaml

To compare the performance of our technique across three distinct languages, we first expressed all benchmarks in Morphic, a new functional language designed with LSS from the ground up. We then implemented semantics-preserving *translators* from Morphic to Standard ML and OCaml. For our baseline measurements, this takes the form of a simple syntactic transformation; Morphic, OCaml, and SML are similar enough that our translations produce acceptably-idiomatic code in all three languages. To measure the effect of applying LSS, we pre-processed those same Morphic source programs using the LSS pass implemented in the Morphic compiler, and then translated the resulting first-order programs to Standard ML and OCaml for further compilation. To evaluate the

benefits of LSS in Morhic, we took as our baseline a version of the Morhic compiler modified to use a more traditional *monovariant* defunctionalization in place of LSS, similar to that of MLton.

One key difference between benchmarks run under Morhic and under OCaml or SML is the treatment of arrays; while Morhic’s lack of mutability forces us to use semantically-immutable arrays in the Morhic benchmarks, our translation to SML and OCaml uses *mutable* arrays for all configurations. We took care when selecting benchmarks to ensure that the mutability of arrays did not affect the benchmark semantics. In fact, Morhic compiler optimizations will choose to convert to mutable arrays for these benchmarks, allowing us to make a more direct comparison.

7.2 Benchmarks

We evaluated the impact of LSS on six different benchmark programs which make use of higher-order functions. All programs were written in Morhic, and tested under MLton and OCaml using automated translators as described in Section 7.1. *Calc* is a calculator utility which parses and evaluates integer arithmetic strings with precedence. *ParseJSON* is a complete JSON deserializer. *PrimesIter* is a program to enumerate and count prime numbers up to a given limit by checking all of their possible divisors by brute force. *PrimesSieve* is an implementation of the sieve of Eratosthenes to enumerate all prime numbers up to a given limit. *Quicksort* sorts arrays of 64-bit integers. *Unify* is a syntactic unification algorithm operating on trees of S-expression-like terms.

We designed our benchmarks to make significant use of *combinator-based library abstractions*, as we believe such abstractions represent a common and important source of higher-order control flow in many functional programs. To this end, we wrote Morhic code to define a lazy iterator library, a state monad library, and a parser combinator library, all making heavy use of higher-order functions in both their implementations and their public APIs. The iterator library is used in the benchmarked sections of *PrimesIter*, *PrimesSieve*, *Quicksort* and *Unify*, the state monad library is used in *Unify*, and the parser combinator library is used in *Calc* and *ParseJSON*. The use of these libraries account for most of the higher-order control flow in our benchmarks.

7.3 Results and Discussion

We found that LSS significantly improved run time performance in all six of our benchmarks—save one—running under all compilers we tested. As shown in Figure 11, speedups under MLton range from 0.91× to 6.85×, and speedups under OCaml range from 1.08× to 3.45×. When compiling our benchmarks with Morhic, the results are more extreme, with specialization enabling speedups of up to 78.93× relative to monovariant defunctionalization. In all compilers, introducing specialized versions of higher-order functions did not significantly increase the size of the generated binary; rather, binary size was not affected by LSS in SML, resulted in *smaller* binaries for Morhic, and resulted in slightly larger binaries for OCaml.

- **MLton:** Our results under MLton show that defunctionalization via LSS can significantly improve performance over monovariant defunctionalization. The MLton compiler incorporates a state-of-the-art monovariant defunctionalization early in its compilation pipeline [Cejtin et al. 2000], and pre-processing with LSS consistently outperforms this monovariant approach across our benchmarks.
- **OCaml:** Our results under OCaml show that LSS can also improve performance over non-defunctionalizing compilation, even when integrated into existing compilers, like OCaml, whose optimization pipelines are not specifically designed to exploit the benefits of defunctionalization. We note that our pre-processing-based LSS implementation increases binary size under OCaml, which we suspect is due to the compiler treating every specialization as a separate externally-linkable symbol.

- **Morphic:** We take our results under Morphic as evidence that LSS can be an especially powerful tool in compilers designed from the beginning to take advantage of it. Morphic's interprocedural analyses for safe in-place mutation and optimized memory management were designed to depend on LSS's specialization for precision, and we see empirically that they do indeed degrade significantly when LSS is replaced with a monovariant defunctionalization.

Across all compilers, we additionally remark that our largest and most realistic benchmark program, *ParseJSON*, is also the program in which we see the largest performance gains due to LSS. We take this as evidence in support of the hypothesis that the benefits of specialization become more significant with increasing scale.

8 RELATED WORK

Defunctionalization was first presented by Reynolds [1972] as a transformation which converts lambda values to instances of single sum type encompassing the representation of each lambda in the program, and calls to higher-order functions to calls to a single apply function which cases on that sum type. Reynolds' defunctionalization can be thought of as conservatively inferring one global lambda set which it applies to indiscriminately to *all* functions in the program. This gave rise to further work on inferring more precise lambda sets using typing information [Bell et al. 1997; Minamide et al. 1996] or control-flow analysis [Cejtin et al. 2000; Dimock et al. 2001]. Of particular note is MLton, a whole-program optimizing compiler for Standard ML which uses control flow analysis [Midtgaard 2012] to infer concrete lambda sets in Standard ML programs. Because MLton's analysis infers concrete lambda sets rather than symbolic lambda set signatures, its analysis is subject to a limitation known as poisoning [Wansbrough and Peyton Jones 1999], whereby the lambda set annotation on the parameter to a higher-order function must approximate the conditions at all of its call-sites in the entire program. While Pottier and Gauthier [2006] propose a polymorphic defunctionalization, this is *not* a *specializing* defunctionalization; rather it is a *monovariant* defunctionalization that handles polymorphic code.

Banerjee [1997] presents an approach to lambda set analysis as polymorphic type inference, but it differs from our inference in key ways which limit its applicability to defunctionalization. In particular, Banerjee leaves inference of μ types to future work, and its use of subtyping means that a program defunctionalized according to its analysis would need to coerce lambdas at runtime between sum types corresponding to differently sized lambda sets (alternatively, such coercions can be avoided by heap-allocating closure data).

Hovgaard [2018] presents a defunctionalization algorithm which specializes higher-order functions with respect to individual lambdas to which they are applied. Like the approach of C++ [Stroustrup 2013, Ch. 26] and Rust [Klabnik and Nichols 2019, Ch. 10] to eliminating higher-order functions during compilation, Hovgaard imposes a type-based restriction on the use of lambdas to prohibit code from dynamically choosing between lambdas, inserting lambdas into arrays, or dynamically nesting closures (handling these cases would require the analysis to reason about lambda sets). Hovgaard highlights that lifting this restriction on the use of lambdas in its source language is a valuable avenue for future work, and anticipates some of the challenges we address in this paper, such as inferring μ types in lambda set annotations.

The array programming language Dex [Paszke et al. 2021] applies a specializing defunctionalization transformation which amounts to inlining higher-order functions anywhere they appear. This is less scalable than LSS, which is capable of deduplicating identical instantiations of high-order functions. Dex does not support general recursion and its rewriting based simplification strategy means that adding general recursion as an extension would be difficult. Similarly, the Julia programming language uses a JIT-based approach to generate precise specializations of each

function the first time it is invoked. This approach does not require lambda sets, as there is no ambiguity at JIT-time about which concrete lambda is being applied; conversely, it requires a JIT. Julia’s combination of multiple-dispatch and hyper-specializing JIT suffers from a form of poisoning in which functions may *become uncallable* at existing call sites due to the discovery of a later specialization [Bezanson et al. 2017].

Where defunctionalization is not available and higher-order functions are heavily used, more incremental techniques for eliminating higher-order functions and analyzing higher-order control flow have proven invaluable for optimization. Bergstrom et al. [2014] present an approach to higher-order inlining for contexts in which the lexical environment at the call-site differs from that at which a closure was constructed, but they explain that their algorithm is unpredictable from the perspective of the developer because the results of its control flow analysis can be thrown off by non-local changes in the program. This unpredictability owes itself to the poisoning property of its underlying control flow analysis, 0-CFA [Midtgaard 2012], which also affects MLton. Jones and Marlow [2002] show the importance of inlining in the context of the Glasgow Haskell Compiler. De Souza [2022] presents the use of function specialization as a means to improve analysis of higher-order functions in INFER, a static analysis framework that deduces preconditions and postconditions of functions to detect potential bugs in Objective-C, C++, and Java programs.

Finally, it is worth considering the similarity between our type system for L^{annot} and existing *effect systems* [Pierce 2005]. With some rearrangement, our L^{annot} typing relation with *constraint context* Q , i.e. $\Delta ; Q ; \Gamma \vdash \epsilon : \tau$, can be viewed alternatively as a typing relation with an *effect* Q , i.e. $\Delta ; \Gamma \vdash \epsilon :^Q \tau$. However, the interaction between our constraint context Q and our lambda abstraction construct does not match the usual formulation of an effect system. In an effect system, the effect annotation for the body of a lambda abstraction is typically independent of the effect annotation for the lambda expression itself; the effects of the body of the lambda are “screened off” from the outer effect context by the lambda abstraction. By contrast, in L^{annot} the constraint contexts for a lambda abstraction and its body are identical, in keeping with the usual treatment of constraint contexts in such systems as Haskell’s typeclass constraints [Wadler and Blott 1989].

9 CONCLUSION

In this paper, we have introduced LSS via an intermediate “annotated” type system which captures precise symbolic information about the possible inhabitants of function values at runtime. We have developed a type inference, formalized in Isabelle/HOL, which we have proven always succeeds in lifting well-typed source programs into our extended type system. We have provided a specialization and lowering procedure that operate on a program typed under this system. Composed with type inference, these procedures form a specializing defunctionalization algorithm that captures both the flexibility of prior monovariant defunctionalization transformations and the performance benefits of prior specializing defunctionalization transformations. Our evaluation confirmed that LSS yields significant performance benefits in practice.

ACKNOWLEDGMENTS

We especially thank Jonathan Ragan-Kelley and Alex Reinking for their valuable guidance and suggestions, as well as Molly Feldman, Ethan Smith, and the anonymous reviewers for their helpful feedback. We thank Sanjit Seshia for supporting the class project that became this paper. In addition to Sutter Hill Ventures and NSF CISE Expeditions Award CCF-1730628, this research is supported by gifts from Amazon Web Services, Astronomer, Ant Group, Ericsson, Futurewei, Google, IBM, Intel, Lacework, Meta, Microsoft, Mohamed Bin Zayed University of Artificial Intelligence, Nexla, Samsung SDS, Scotiabank, Uber, and VMware.

ARTIFACT AVAILABILITY

Our Isabelle/HOL formalism as well as the source code of the Morphic compiler and our benchmarks (listed in Figure 11) have been packaged into a Docker image which is available via Zenodo [Brandon et al. 2023b]. They are respectively contained in directories `/home/morphic/LSSIIsabelle` and `/home/morphic/morphic` of the Docker image alongside `README.md` files detailing how to reproduce our results.

REFERENCES

- Anindya Banerjee. 1997. A Modular, Polyvariant and Type-Based Closure Analysis. In *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming* (Amsterdam, The Netherlands) (ICFP '97). Association for Computing Machinery, New York, NY, USA, 1–10. <https://doi.org/10.1145/258948.258951>
- Jeffrey M Bell, Françoise Bellegarde, and James Hook. 1997. Type-driven defunctionalization. In *Proceedings of the second ACM SIGPLAN international conference on Functional programming*. 25–37. <https://doi.org/10.1145/258948.258953>
- Lars Bergstrom, Matthew Fluet, Matthew Le, John Reppy, and Nora Sandler. 2014. Practical and Effective Higher-Order Optimizations. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming* (Gothenburg, Sweden) (ICFP '14). Association for Computing Machinery, New York, NY, USA, 81–93. <https://doi.org/10.1145/2628136.2628153>
- Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A fresh approach to numerical computing. *SIAM review* 59, 1 (2017), 65–98. <https://doi.org/10.1137/141000671>
- William Brandon, Benjamin Driscoll, Frank Dai, and Wilson Berkow. 2023a. Morphic Research Language. <https://morphic-lang.org> [Online].
- William Brandon, Benjamin Driscoll, Frank Dai, Wilson Berkow, and Mae Milano. 2023b. *Better Defunctionalization Through Lambda Set Specialization*. <https://doi.org/10.5281/zenodo.7712285>
- Henry Cejtin, Suresh Jagannathan, and Stephen Weeks. 2000. Flow-directed closure conversion for typed languages. In *European Symposium on Programming*. Springer, Berlin, Heidelberg, 56–71. https://doi.org/10.1007/3-540-46425-5_4
- Corentin De Souza. 2022. Higher-order function specialization in Infer. Presented at the 43rd ACM SIGPLAN Conference on Programming Language Design and Implementation.
- Allyn Dimock, Ian Westmacott, Robert Muller, Franklyn Turbak, and Joseph Brian Wells. 2001. Functioning without closure: Type-safe customized function representations for Standard ML. In *Proceedings of the sixth ACM SIGPLAN international conference on Functional programming*. 14–25. <https://doi.org/10.1145/507635.507640>
- Roger Hindley. 1969. The Principal Type-Scheme of an Object in Combinatory Logic. *Trans. Amer. Math. Soc.* 146 (1969), 29–60. <https://doi.org/10.2307/1995158>
- Anders Kiel Hovgaard. 2018. *Higher-order functions for a high-performance programming language for GPUs*. Master's thesis. University of Copenhagen, Copenhagen, Denmark.
- Anders Kiel Hovgaard, Troels Henriksen, and Martin Elsman. 2018. High-Performance Defunctionalisation in Futhark. In *International Symposium on Trends in Functional Programming*. Springer, Cham, Switzerland, 136–156. https://doi.org/10.1007/978-3-030-18506-0_7
- Simon Peyton Jones and Simon Marlow. 2002. Secrets of the Glasgow Haskell Compiler inliner. *Journal of Functional Programming* 12, 4-5 (2002), 393–434. <https://doi.org/10.1017/S0956796802004331>
- Steve Klabnik and Carol Nichols. 2019. *The Rust Programming Language*. No Starch Press, San Francisco, CA.
- Xavier Leroy, Damien Doligez, Alain Frisch, Jacques Garrigue, Didier Rémy, and Jérôme Vouillon. 2022. The OCaml system release 4.14: Documentation and user's manual. (2022).
- Jan Midtgaard. 2012. Control-flow analysis of functional programs. *ACM computing surveys (CSUR)* 44, 3 (2012), 1–33. <https://doi.org/10.1145/2187671.2187672>
- Robin Milner. 1978. A theory of type polymorphism in programming. *Journal of computer and system sciences* 17, 3 (1978), 348–375. [https://doi.org/10.1016/0022-0000\(78\)90014-4](https://doi.org/10.1016/0022-0000(78)90014-4)
- Yasuhiro Minamide, Greg Morrisett, and Robert Harper. 1996. Typed closure conversion. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT symposium on principles of programming languages*. 271–283. <https://doi.org/10.1145/237721.237791>
- Adam Paszke, Daniel Johnson, David Duvenaud, Dimitrios Vytiniotis, Alexey Radul, Matthew Johnson, Jonathan Ragan-Kelley, and Dougal Maclaurin. 2021. Getting to the point. Index sets and parallelism-preserving autodiff for pointful array programming. *arXiv preprint arXiv:2104.05372* (2021). <https://doi.org/10.1145/3473593>
- Benjamin C Pierce. 2002. *Types and programming languages*. MIT Press, Cambridge, MA.
- Benjamin C Pierce. 2005. *Advanced topics in types and programming languages*. MIT Press, Cambridge, MA.
- François Pottier and Nadji Gauthier. 2006. Polymorphic typed defunctionalization and concretization. *Higher-Order and Symbolic Computation* 19, 1 (2006), 125–162. <https://doi.org/10.1007/s10990-006-8611-7>

- John C Reynolds. 1972. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM annual conference - Volume 2*. ACM, New York, NY, USA, 717–740. <https://doi.org/10.1145/800194.805852>
- Bjarne Stroustrup. 2013. *The C++ Programming Language*. Addison-Wesley, Boston, MA.
- Philip Wadler and Stephen Blott. 1989. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 60–76. <https://doi.org/10.1145/75277.75283>
- Keith Wansbrough and Simon Peyton Jones. 1999. Once upon a polymorphic type. In *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 15–28. <https://doi.org/10.1145/292540.292545>
- Stephen Weeks. 2006. Whole-program compilation in MLton. *ML* 6 (2006), 1–1. <https://doi.org/10.1145/1159876.1159877>
- A.K. Wright. 1995. Simple imperative polymorphism. *LISP and Symbolic Computation* 8 (1995), 343–355. <https://doi.org/10.1007/BF01018828>

Received 2022-11-10; accepted 2023-03-31